FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	SANTOS, TAYSIAN
Section:	AN-43
Professor:	MISS TAN
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

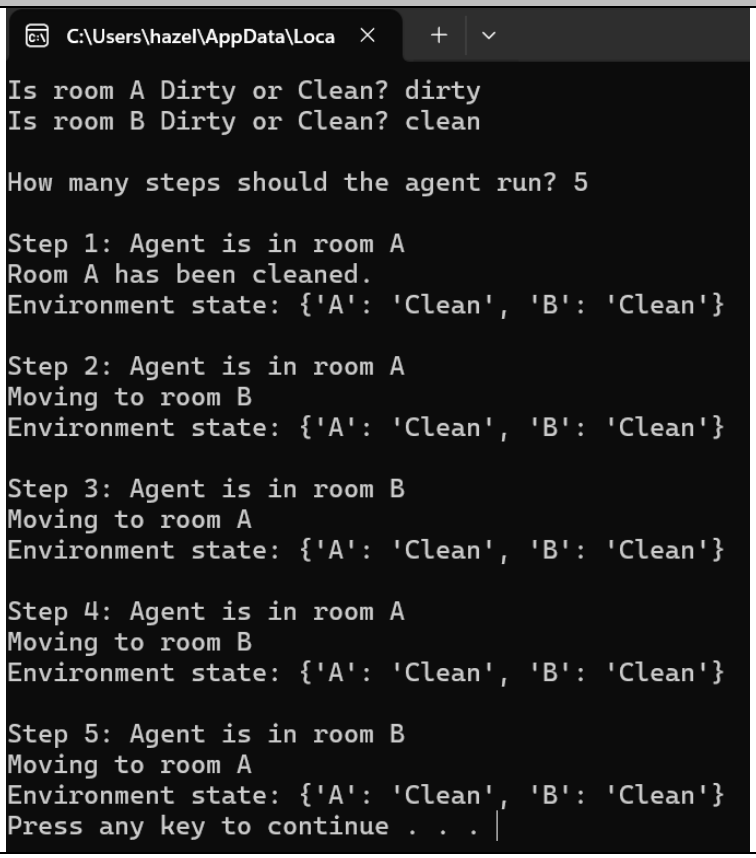
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

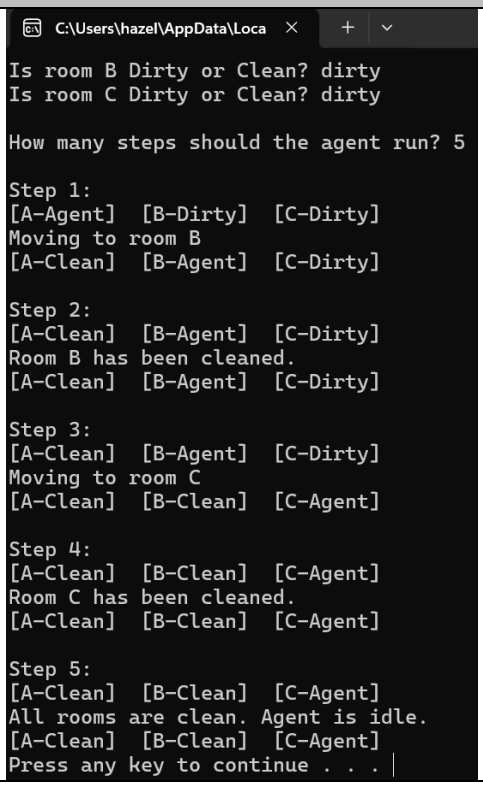
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
Rule-Based Room Cleaning Agent
Number of rooms (2 or 3) [2]: 3
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean
Is room C Dirty or Clean? dirty
How many steps should the agent run? 5

Initial environment:
[A-Agent] [B-Clean] [C-Dirty]
Environment state: {'A': 'Dirty', 'B': 'Clean', 'C': 'Dirty'}

Step 1: Agent is in room A
Room A has been cleaned.
[A-Agent] [B-Clean] [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Step 2: Agent is in room A
Moving to room C.
[A-Clean] [B-Clean] [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Step 3: Agent is in room C
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 4: Agent is in room C
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Simulation complete.
Final location: room C
Final environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
```

```
"""Two-room rule-based cleaning agent (with optional three-room extension).
Run: python TA2_Santos.py
No external packages are required.
"""

import random


class CleaningAgent:
    """Perceive the current room and clean it or move to a dirty room."""

    def __init__(self, environment, location="A"):
        self.environment = environment
        self.location = location

    def move(self, destination):
        """Move to another room in the environment."""
        if destination not in self.environment:
```

```
            raise ValueError(f"Unknown room: {destination}")
        self.location = destination

    def perceive_and_act(self):
        """Apply the rules once and return a description of the action."""
        if self.environment[self.location] == "Dirty":
            self.environment[self.location] = "Clean"
            return f"Room {self.location} has been cleaned."

        dirty_rooms = [
            room
            for room, condition in self.environment.items()
            if condition == "Dirty" and room != self.location
        ]
        if dirty_rooms:
            destination = random.choice(dirty_rooms)
            self.move(destination)
            return f"Moving to room {destination}."

        return "All rooms are clean. Agent is idle."


def ask_room_condition(room):
    """Ask for a room state and validate the answer."""
    while True:
        answer = input(f"Is room {room} Dirty or Clean? ").strip().lower()
        if answer in ("dirty", "clean"):
            return answer.title()
        print("Please enter Dirty or Clean.")


def ask_room_count():
    """Choose the required two-room task or the optional three-room extension."""
    while True:
        answer = input("Number of rooms (2 or 3) [2]: ").strip()
        if not answer:
            return 2
        if answer in ("2", "3"):
            return int(answer)
        print("Please enter 2 or 3.")


def ask_steps():
    """Read a non-negative number of simulation steps."""
```

```
    while True:
        answer = input("How many steps should the agent run? ").strip()
        try:
            steps = int(answer)
            if steps >= 0:
                return steps
        except ValueError:
            pass
        print("Please enter a whole number zero or greater.")


def show_environment(environment, agent):
    """Print a text-based room grid and the complete room-state dictionary."""
    cells = []
    for room, condition in environment.items():
        label = f"{room}-Agent" if room == agent.location else f"{room}-{condition}"
        cells.append(f"[{label}]")
    print(" ".join(cells))
    print(f"Environment state: {environment}")


def main():
    print("Rule-Based Room Cleaning Agent")
    room_count = ask_room_count()
    room_names = ["A", "B", "C"][:room_count]
    environment = {room: ask_room_condition(room) for room in room_names}
    steps = ask_steps()

    # Fixed seed makes the random destination choice reproducible.
    random.seed(7)
    agent = CleaningAgent(environment, location="A")
    print("\nInitial environment:")
    show_environment(environment, agent)

    for step_number in range(1, steps + 1):
        print(f"\nStep {step_number}: Agent is in room {agent.location}")
        action = agent.perceive_and_act()
        print(action)
        show_environment(environment, agent)
        if action == "All rooms are clean. Agent is idle.":
            break

    print("\nSimulation complete.")
    print(f"Final location: room {agent.location}")
```

```
    print(f"Final environment: {environment}")


if __name__ == "__main__":
    main()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. **If its current room is Dirty, it cleans that room. If the current room is Clean and another room is Dirty, it moves to a dirty room. When no dirty rooms remain, it stays in place and reports that it is idle.**
2. **It checks the condition of the room it currently occupies. Dirty means clean; Clean means look for a different dirty room and move there. It does not move away from a room that is dirty before cleaning it.**
3. **The agent starts in room A, finds it clean, and finds no dirty destination. It reports that all rooms are clean and become idle without changing either room.**
4. **Storeroom names and conditions in a dictionary or list and represent connections as a graph if rooms are not in a line. When the current room is clean, find reachable dirty rooms and choose a destination. The included three-room extension uses a dictionary and randomly selects among other dirty rooms**